

**UC San Diego**

Electrical and Computer Engineering

**JACOBS SCHOOL OF ENGINEERING**



# Variational Autoencoders

ECE 285 Deep Generative Models

Youwei Liang

# Today's Topic

- Variational Autoencoders (VAEs)
  - Recap
  - Mathematical details
  - Implementations
  - Variants of VAE

# Recap of Latent Variables

- Generating face images
- Needs a simple distribution to sample from
  - Normal (Gaussian)
  - Categorical
- It is called latent distribution
  - Modeling factors like race, gender, eye color, hair color, etc
- Latent (not observed) variables  $z$
- Data: images  $x$
- Encoder
  - Learn a model from  $x$  to  $z$
- Decoder
  - Learn a model from  $z$  to  $x$



# Recap of Evidence Lower Bound

- The unbiased estimator of the log likelihood of our data  $\mathbf{x}$

$$\log \left( \sum_{\mathbf{z} \in \mathcal{Z}} p_{\theta}(\mathbf{x}, \mathbf{z}) \right) = \log \left( \sum_{\mathbf{z} \in \mathcal{Z}} \frac{q(\mathbf{z})}{q(\mathbf{z})} p_{\theta}(\mathbf{x}, \mathbf{z}) \right) = \log \left( \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} \left[ \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \right] \right)$$

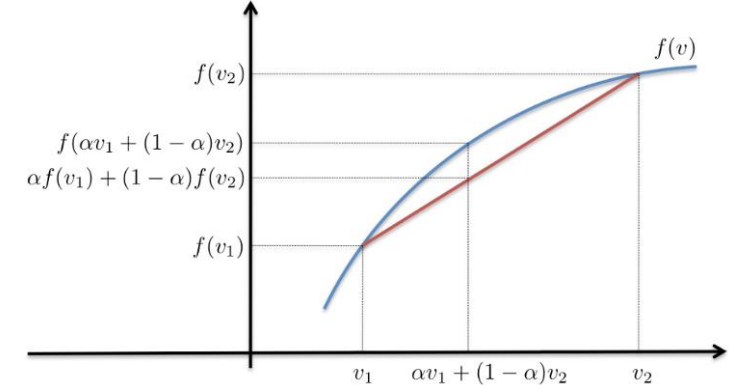
- Using Jensen's inequality for concave functions (log)

$$\log(\mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} [f(\mathbf{z})]) = \log\left(\sum_{\mathbf{z}} q(\mathbf{z}) f(\mathbf{z})\right) \geq \sum_{\mathbf{z}} q(\mathbf{z}) \log f(\mathbf{z}) = \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} [\log f(\mathbf{z})]$$

- Maximize the **Evidence Lower Bound (ELBO)**

$$\log \left( \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} \left[ \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \right] \right) \geq \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} \left[ \log \left( \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \right) \right]$$

ELBO



# When Evidence Lower Bound Is Tight

- Evidence lower bound (ELBO) holds for **any**  $q(z)$  over the hidden variables

$$p_{\theta}(\mathbf{x}) = \log \left( \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} \left[ \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \right] \right) \geq \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} \left[ \log \left( \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \right) \right]$$

- Equality holds if  $q(z) = p(z|x)$

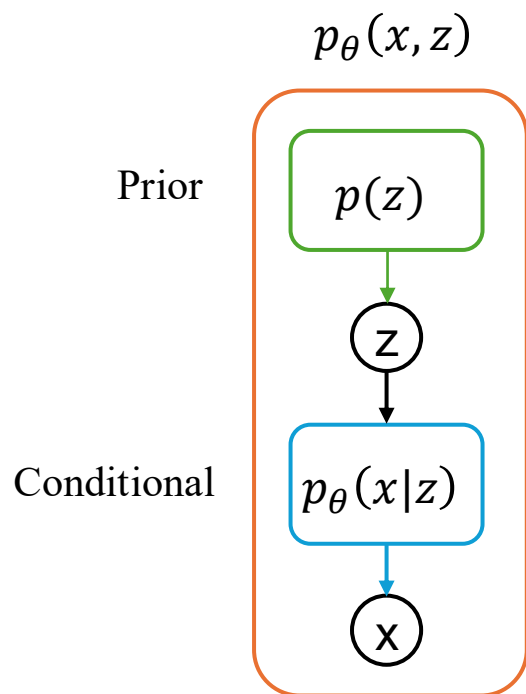
$$\begin{aligned} \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}) \log \frac{p(\mathbf{x}, \mathbf{z}; \theta)}{p(\mathbf{z}|\mathbf{x})} &= \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}) \log \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x}; \theta)}{p(\mathbf{z}|\mathbf{x})} \\ &= \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}) \log p(\mathbf{x}; \theta) \\ &= \log p(\mathbf{x}; \theta) \underbrace{\sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x})}_{=1} \\ &= \log p(\mathbf{x}; \theta) \end{aligned}$$

- The true posterior  $p(z|x) = p_{\theta}(x|z)p_{\theta}(z)/p_{\theta}(x)$  (Bayes' theorem)
  - Intractable to compute
  - Learn a model to approximate the true posterior

# ELBO and the Prior

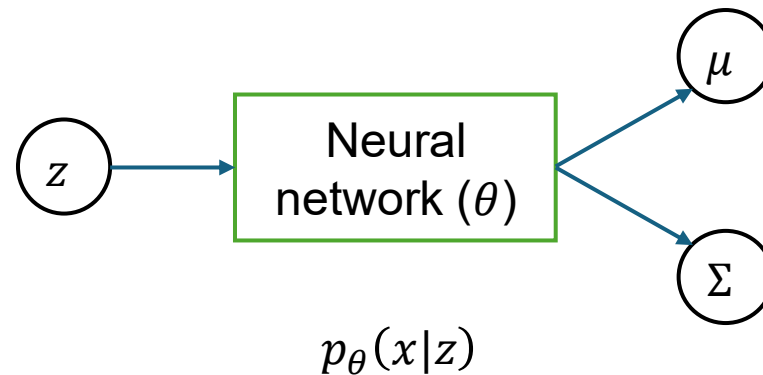
- Using a (yet-to-learn) posterior  $q_\phi(z|x)$  in place of  $q(z)$
- How about the joint  $p(x, z)$

$$\begin{aligned}\log p_\theta(\mathbf{x}^{(i)}) &\geq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [-\log q_\phi(\mathbf{z}|\mathbf{x}) + \log p_\theta(\mathbf{x}, \mathbf{z})] \\ &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [-\log q_\phi(\mathbf{z}|\mathbf{x}) + \log p_\theta(\mathbf{x}|\mathbf{z}) + \log p(\mathbf{z})] \\ &= -\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[ \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right] + \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] \\ &= -D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) + \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]\end{aligned}$$



# Define the Generative Process

- We already see the prior  $p(z)$  can be any distribution (we don't have to learn it)
  - Recall  $p_{\theta}(x, z) = p_{\theta}(z)p_{\theta}(x|z)$
  - For simplicity, we choose the standard normal distribution as the prior:  $p_{\theta}(z) = \mathcal{N}(0, I)$
  - Since the prior is fixed, we focus on learning the conditional distribution  $p_{\theta}(x|z)$
- How to parameterize  $p_{\theta}(x|z)$ 
  - Distribution generating model
    - $p_{\theta}(x|z) = \mathcal{N}(x; \mu(z), \Sigma(z))$
    - A neural network parameterized by  $\theta$
    - Output: mean and covariance of a normal distribution

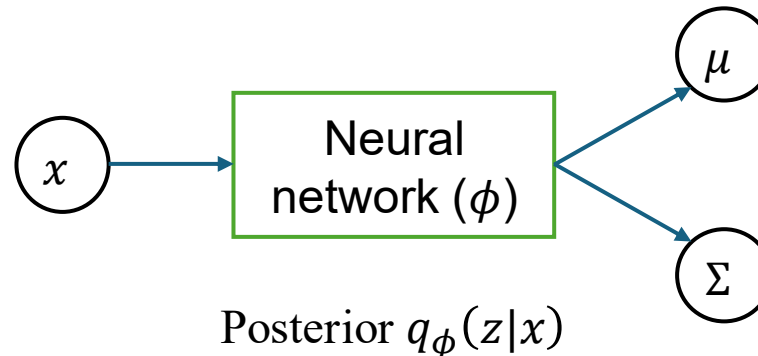


# Approximating the Posterior

- We have chosen the standard normal as the prior:  $p_\theta(z) = \mathcal{N}(0, I)$
- The auxiliary distribution  $q$  can be any distribution over the hidden variables  $z$
- We want to use a  $q$  so that it is close to the true posterior distribution  $q_\phi(z) \approx p(z|x)$ 
  - Reason #1: in this case the bound is tight (exact optimization of the log likelihood of the data  $x$ )
  - Reason #2: a latent  $z$  sampled from  $p(z|x)$  is more meaningful (variance reduction in importance sampling)
  - Intuition: the training should focus on the latent variables that are closely related to the data point  $x$

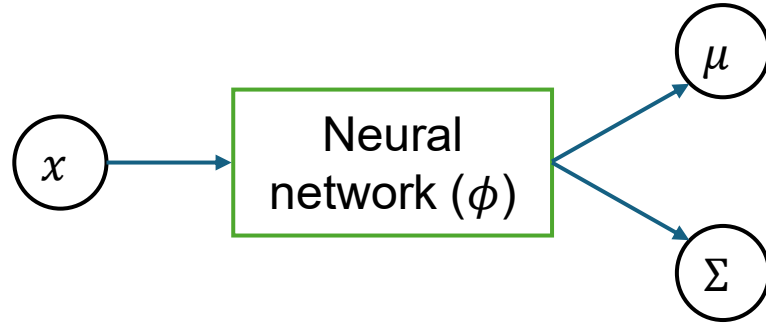
$$\log \left( \mathbb{E}_{z \sim q(z)} \left[ \frac{p_\theta(\mathbf{x}, z)}{q(z)} \right] \right) \geq \mathbb{E}_{z \sim q(z)} \left[ \log \left( \frac{p_\theta(\mathbf{x}, z)}{q(z)} \right) \right]$$

- For each  $x$ , we can have a different  $q$
- We use a distribution generating model for  $q_\phi(z|x) = \mathcal{N}(z; \mu(x), \Sigma(x))$ 
  - A neural network parameterized by  $\phi$
  - Output: mean and covariance of a normal distribution

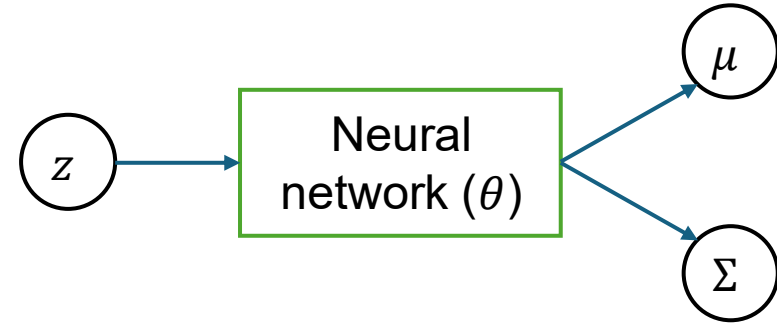


# Training Objective

- We now have parameterized the posterior distribution and generative distribution



Learn the posterior  $q_{\phi}(z|x)$



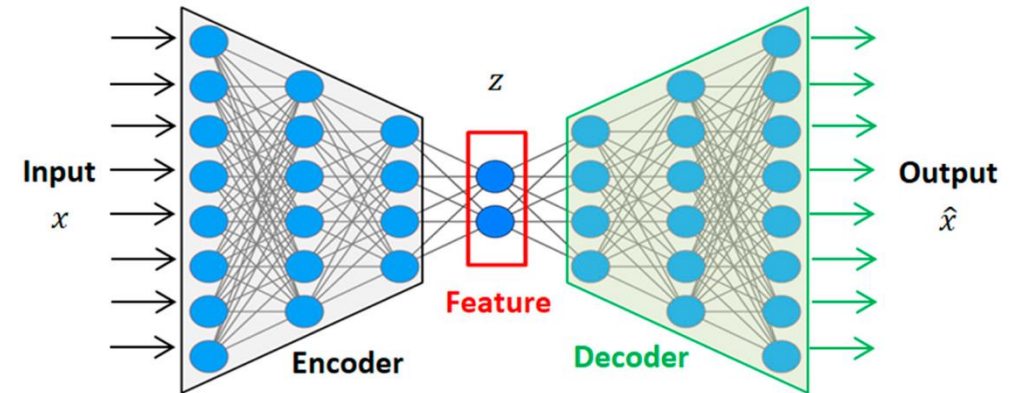
Learn the generative  $p_{\theta}(x|z)$

- Recall our training objective ELBO

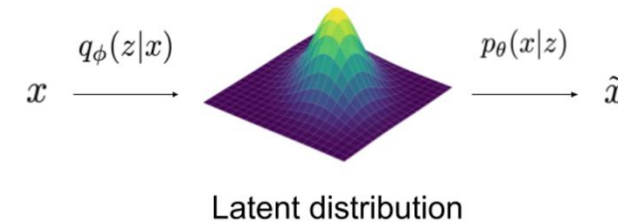
$$\log p_{\theta}(\mathbf{x}^{(i)}) \geq \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)}) = -D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})]$$

# Intuition Behind Training Variational Autoencoders

- Autoencoders
  - Training with a reconstruction loss
  - No restriction on the latents
  - Useful for learning data features



- Variational autoencoders (VAEs)
  - Training with ELBO
  - Regularize the latent distribution with a predefined prior
  - Useful for generation

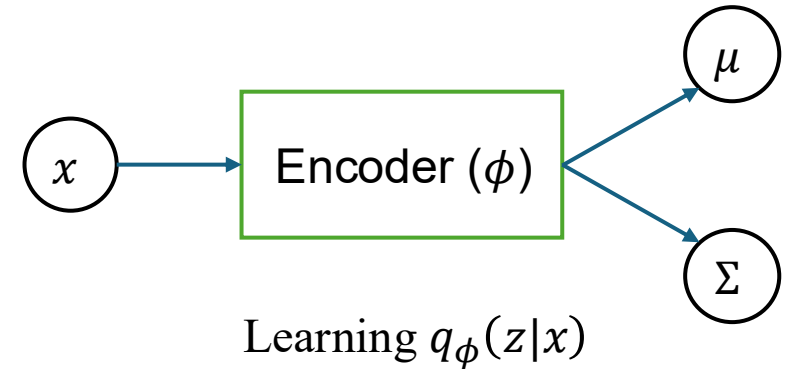


$$\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) = -D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) + \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]$$

- Train VAEs with gradient ascent
  - Update  $\phi$  and  $\theta$  with mini-batch samples

# Update of the Encoder

- Use a convolutional neural network (CNN) to encode images
  - We assume a diagonal covariance matrix  $\Sigma$
  - Input: an image  $x$
  - Output: two vectors: mean  $\mu$ , and variance  $\Sigma$
- Calculate the gradient of the KL divergence w.r.t.  $\phi$



$$\begin{aligned} & -D_{KL}(q_{\phi}(z|x)||p(z)) \\ &= -\int q_{\phi}(z|x) \log q_{\phi}(z|x) dz + \int q_{\phi}(z|x) \log p(z) dz \end{aligned}$$

$$\begin{aligned} \int q_{\theta}(z) \log q_{\theta}(z) dz &= \int \mathcal{N}(z; \mu, \sigma^2) \log \mathcal{N}(z; \mu, \sigma^2) dz \\ &= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^J (1 + \log \sigma_j^2) \end{aligned}$$

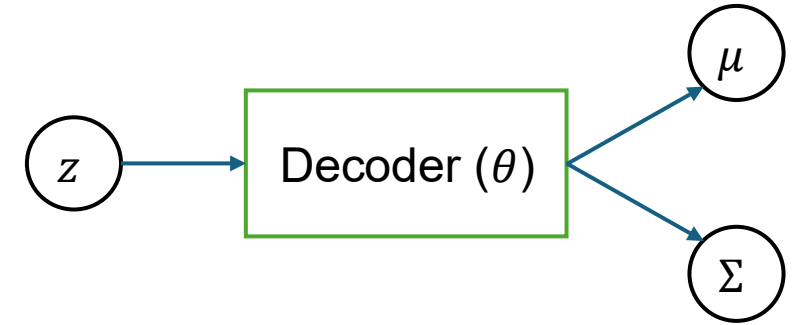
$$\begin{aligned} \int q_{\theta}(z) \log p(z) dz &= \int \mathcal{N}(z; \mu, \sigma^2) \log \mathcal{N}(z; \mathbf{0}, \mathbf{I}) dz \\ &= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^J (\mu_j^2 + \sigma_j^2) \end{aligned}$$

$$-D_{KL}(q_{\phi}(z|x)||p(z)) = \frac{1}{2} \sum_{j=1}^d (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2)$$

How close is  $q_{\phi}(z|x)$  to standard normal  $\mathcal{N}(0, I)$

# Update of the Decoder

- Use a neural network with multiple deconvolutional (transposed convolutional) layers
  - $p_{\theta}(x|z) = \mathcal{N}(x; \mu(z), \Sigma(z))$
  - We assume a diagonal covariance matrix  $\Sigma$
  - Input: a latent vector
  - Output: two tensors (same shape as the image): mean  $\mu$ , and variance  $\Sigma$
  - Each element in  $\mu$  and  $\Sigma$  corresponds to a single pixel in the image



Learning  $p_{\theta}(x|z)$

$$\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) = -D_{KL}(q_{\phi}(z|\mathbf{x})||p(z)) + \mathbb{E}_{q_{\phi}(z|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|z)]$$

- How to derive the gradient of ELBO w.r.t  $\theta$ ?

$$\begin{aligned} \log p(\mathbf{x}|\mathbf{z}) &= \log \left[ \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp \left( -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right) \right] \\ &= \log \left[ \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \right] - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \\ &= -\frac{1}{2} [d \log(2\pi) + \log |\Sigma|] - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \\ &= -\frac{1}{2} \left[ d \log(2\pi) + \sum_{i=1}^d \log \sigma_i^2 + \sum_{i=1}^d \frac{(x_i - \mu_i)^2}{\sigma_i^2} \right]. \end{aligned}$$

Reconstruction loss

# Reparameterization Trick

- How to derive the gradient of ELBO w.r.t  $\phi$ ?

- REINFORCE algorithm

$$\begin{aligned}\nabla_{\phi} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) &= \nabla_{\phi} \int_{\mathbf{z}} q_{\phi}(\mathbf{z}|\mathbf{x}) \log p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z} \\ &= \int_{\mathbf{z}} \log p_{\theta}(\mathbf{x}|\mathbf{z}) \nabla_{\phi} q_{\phi}(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\ &= \int_{\mathbf{z}} \log p_{\theta}(\mathbf{x}|\mathbf{z}) \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \nabla_{\phi} q_{\phi}(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\ &= \int_{\mathbf{z}} \log p_{\theta}(\mathbf{x}|\mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x}) \nabla_{\phi} \log q_{\phi}(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\ &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z}) \nabla_{\phi} \log q_{\phi}(\mathbf{z}|\mathbf{x})]\end{aligned}$$

- Reparameterization of  $q_{\phi}(\mathbf{z}|\mathbf{x})$

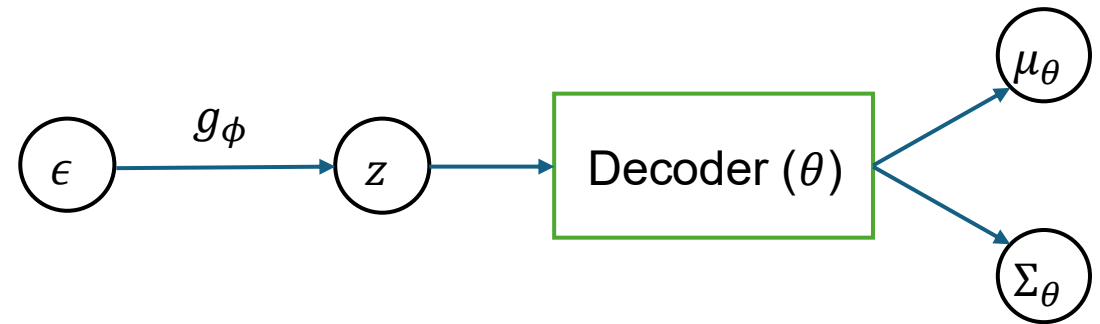
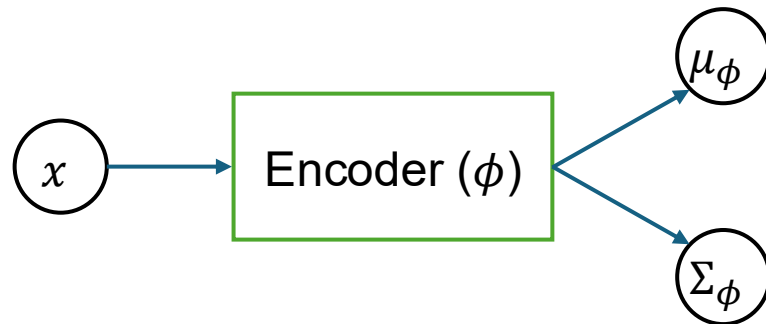
- Sample from a fixed distribution  $p(\epsilon)$ , and map  $\epsilon$  to  $\mathbf{z}$

$$\begin{aligned}\epsilon &\sim p(\epsilon), & \epsilon &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ \mathbf{z} &= g_{\phi}(\epsilon; \mathbf{x}) & \mathbf{z} &= \Sigma_{\phi} \epsilon + \mu_{\phi}\end{aligned}$$

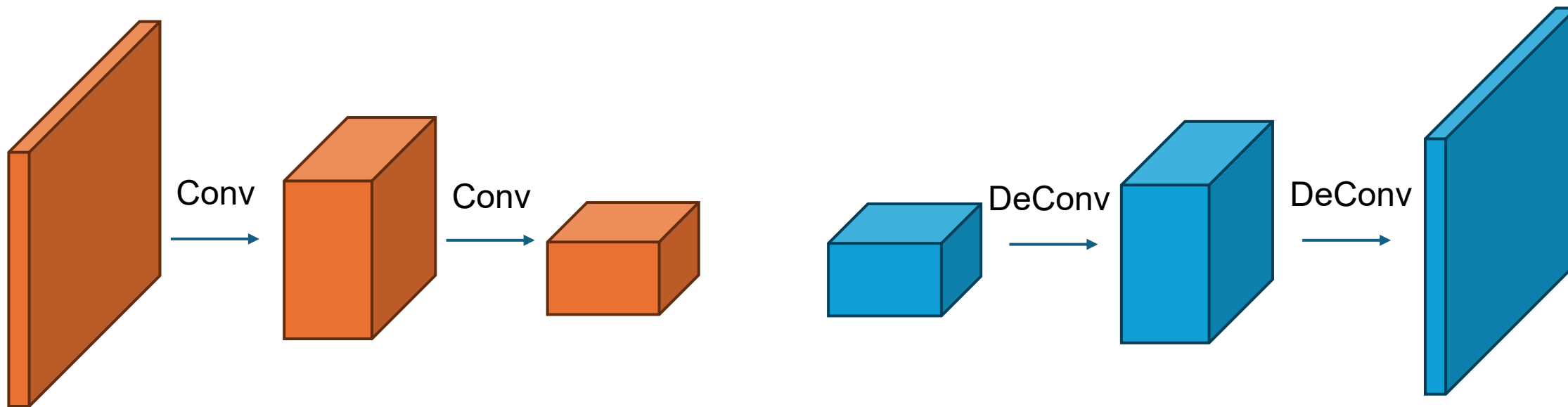
$$\begin{aligned}\nabla_{\phi} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) &= \nabla_{\phi} \mathbb{E}_{p(\epsilon)} \log p_{\theta}(\mathbf{x}|g_{\phi}(\epsilon; \mathbf{x})) && \text{Law of the unconscious statistician} \\ &= \mathbb{E}_{p(\epsilon)} \nabla_{\phi} \log p_{\theta}(\mathbf{x}|g_{\phi}(\epsilon; \mathbf{x})) \\ &\approx \frac{1}{m} \sum_{i=1}^m \nabla_{\phi} \log p_{\theta}(\mathbf{x}|g_{\phi}(\epsilon_i; \mathbf{x})), \quad \text{where } \epsilon_i \sim p(\epsilon).\end{aligned}$$

# Training

- Randomly draw a data point  $x$  from dataset
- Use the encoder to compute the mean  $\mu_\phi$  and covariance  $\Sigma_\phi$  of posterior distribution  $q_\phi(z|x)$
- Randomly draw  $\epsilon$  from standard normal distribution
- Compute the latent  $z = g_\phi(\epsilon) = \Sigma_\phi \epsilon + \mu_\phi$
- Use the decoder to compute the predicted image's mean  $\mu_\theta$  and covariance  $\Sigma_\theta$
- Compute the ELBO
  - Use  $\mu_\phi$  and  $\Sigma_\phi$  to compute the KL divergence  $D_{KL}(q_\phi(z|x)||p(z))$
  - Use  $\mu_\theta$  and  $\Sigma_\theta$  to compute the reconstruction error  $\mathbb{E}_{p(\epsilon)}[p_\theta(x|g_\phi(\epsilon))]$
  - Perform gradient ascent with  $-D_{KL}(q_\phi(z|x)||p(z)) + \mathbb{E}_{p(\epsilon)}[p_\theta(x|g_\phi(\epsilon))]$



# Feature Maps



# Architectures

## Encoder

```
1  import torch
2  import torch.nn as nn
3
4  in_channels = 3
5  hidden_dims = [32, 64, 128, 256, 512]
6  latent_dim = 1024
7
8  modules = []
9  for h_dim in hidden_dims:
10     modules.append(
11         nn.Sequential(
12             nn.Conv2d(in_channels, out_channels=h_dim,
13                       kernel_size=3, stride=2, padding=1),
14             nn.BatchNorm2d(h_dim),
15             nn.LeakyReLU())
16     )
17     in_channels = h_dim
18
19
20 encoder = nn.Sequential(*modules)
21 fc_mu = nn.Linear(hidden_dims[-1]*4, latent_dim)
22 fc_var = nn.Linear(hidden_dims[-1]*4, latent_dim)
23
24 img = torch.randn(1, 3, 64, 64)
25 latent = encoder(img)
26 latent = torch.flatten(latent, start_dim=1)
27 mu = fc_mu(latent)
28 log_var = fc_var(latent)
```

## Loss:

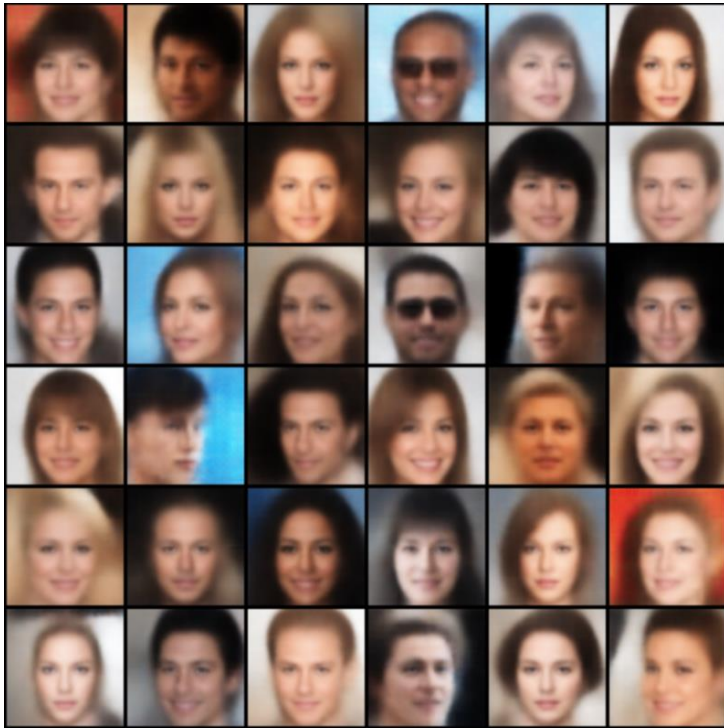
```
64 recons_loss = F.mse_loss(reconst, img)
65 kld_loss = torch.mean(-0.5 * torch.sum(1 + log_var - mu ** 2 - log_var.exp(), dim=1), dim=0)
66 loss = recons_loss + 0.001 * kld_loss
```

## Decoder

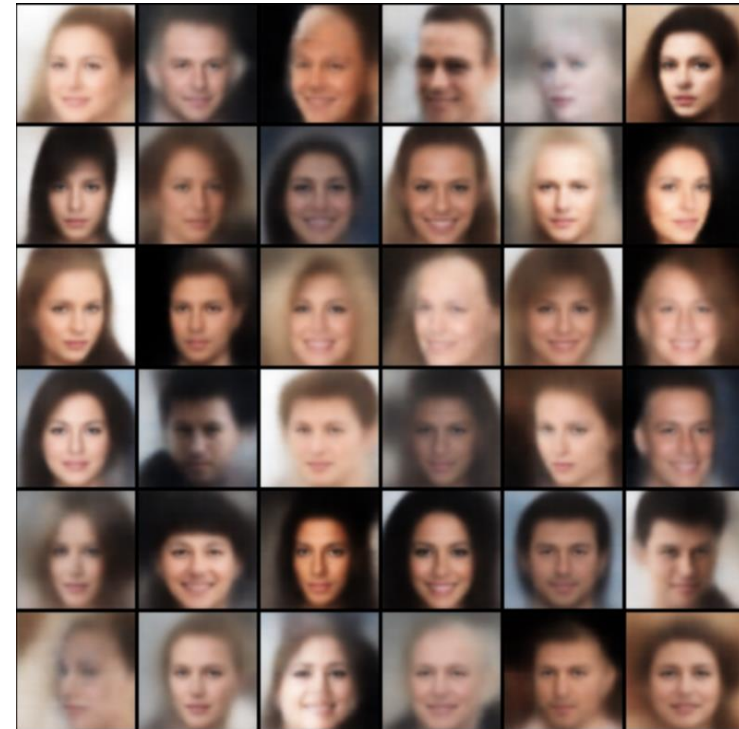
```
30 decoder_input = nn.Linear(latent_dim, hidden_dims[-1] * 4)
31 modules = []
32 hidden_dims.reverse()
33 for i in range(len(hidden_dims) - 1):
34     modules.append(
35         nn.Sequential(
36             nn.ConvTranspose2d(hidden_dims[i],
37                                hidden_dims[i + 1],
38                                kernel_size=3,
39                                stride=2,
40                                padding=1,
41                                output_padding=1),
42             nn.BatchNorm2d(hidden_dims[i + 1]),
43             nn.LeakyReLU())
44     )
45 modules.append(
46     nn.Sequential(
47         nn.Conv2d(hidden_dims[-1], out_channels=3,
48                   kernel_size=3, padding=1),
49         nn.Tanh())
50     )
51 )
52 decoder = nn.Sequential(*modules)
53 std = torch.exp(0.5 * log_var)
54 eps = torch.randn_like(std)
55 z = eps * std + mu
56 result = decoder_input(z)
57 result = result.view(-1, 512, 2, 2)
58 result = decoder(result)
```

# Inference

- Generate novel images
  - Sample a random noise from standard normal:  $\epsilon \sim \mathcal{N}(0, I)$
  - Use the decoder to map  $\epsilon$  to mean and standard deviation
  - Usually, we directly take the mean as the generated image



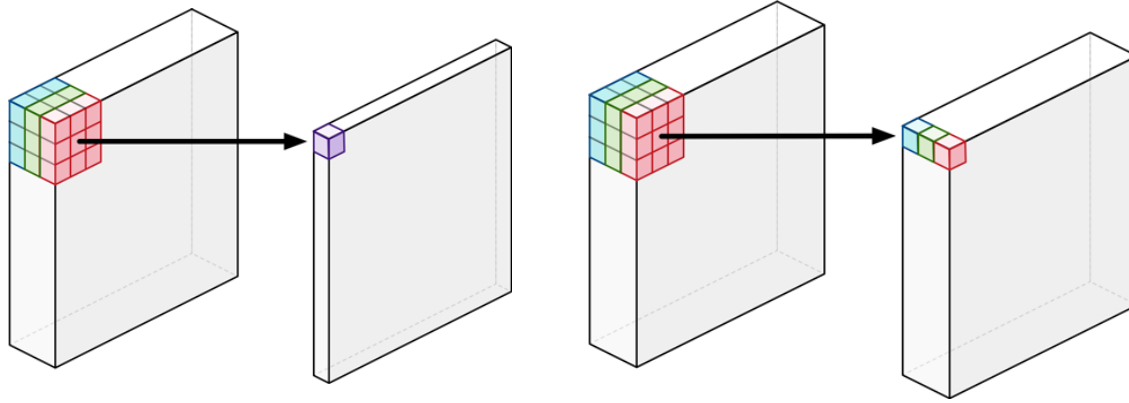
Reconstruction



Samples

# Nouveau VAE (NVAE)

- Architectural advancement
  - Residual cell + depth-wise separable convolution

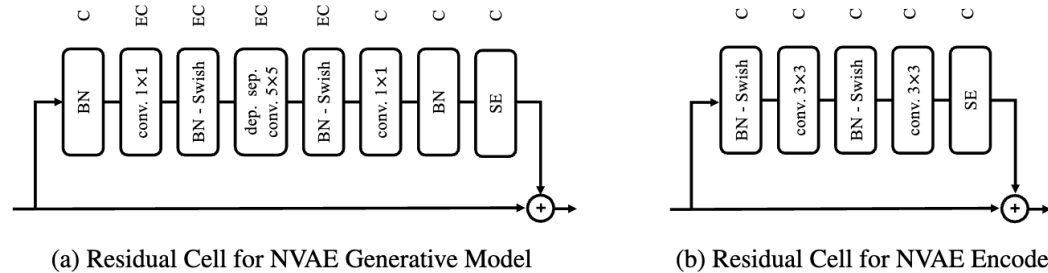


Regular Convolution

Depth-wise Convolution

```
class depthwise_separable_conv(nn.Module):
    def __init__(self, nin, kernels_per_layer, nout):
        super(depthwise_separable_conv, self).__init__()
        self.depthwise = nn.Conv2d(
            nin, nin * kernels_per_layer, kernel_size=3, padding=1, groups=nin)
        self.pointwise = nn.Conv2d(nin * kernels_per_layer, nout, kernel_size=1)

    def forward(self, x):
        out = self.depthwise(x)
        out = self.pointwise(out)
        return out
```



(a) Residual Cell for NVAE Generative Model

(b) Residual Cell for NVAE Encoder

Figure 3: The NVAE residual cells for generative and encoder models are shown in (a) and (b). The number of output channels is shown above. The residual cell in (a) expands the number of channels  $E$  times before applying the depthwise separable convolution, and then maps it back to  $C$  channels. The cell in (b) applies two series of BN-Swish-Conv without changing the number of channels.

# Nouveau VAE (NVAE)

- Architectural advancement
  - Batch Normalization (BN)
    - Regularization on the norm of scaling parameters in BN layers → not amply noise
  - Swish Activation + BN
    - Recall VAE used leaky ReLU

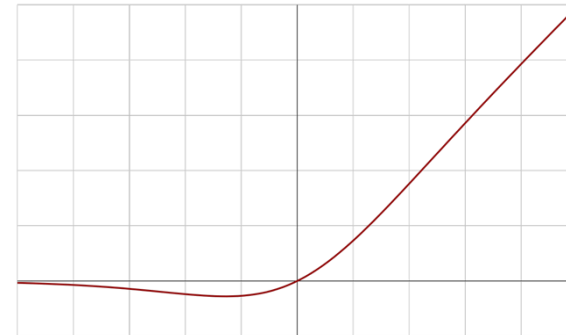
**Input:** Values of  $x$  over a mini-batch:  $\mathcal{B} = \{x_{1\dots m}\}$ ;  
Parameters to be learned:  $\gamma, \beta$   
**Output:**  $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

**Algorithm 1:** Batch Normalizing Transform, applied to activation  $x$  over a mini-batch.

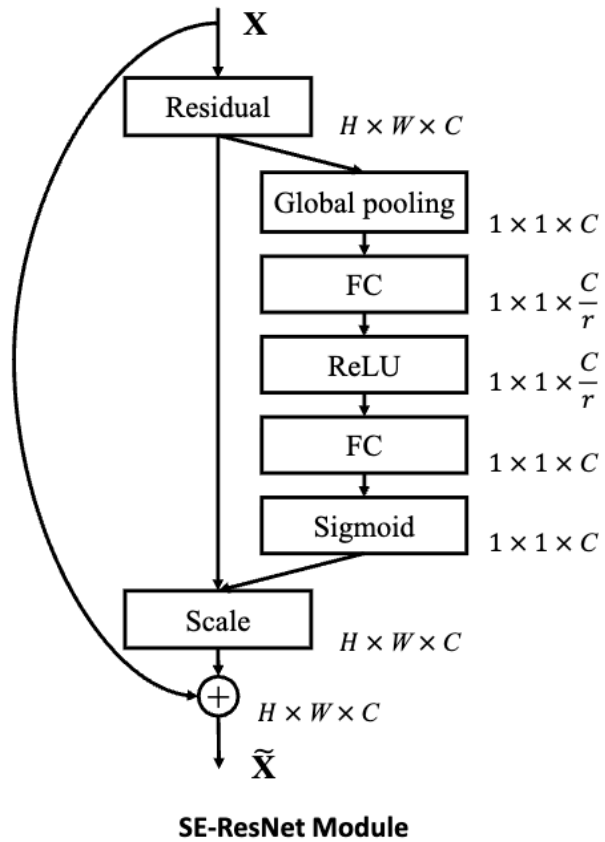
Swish Activation

$$f(u) = \frac{u}{1+e^{-u}}$$



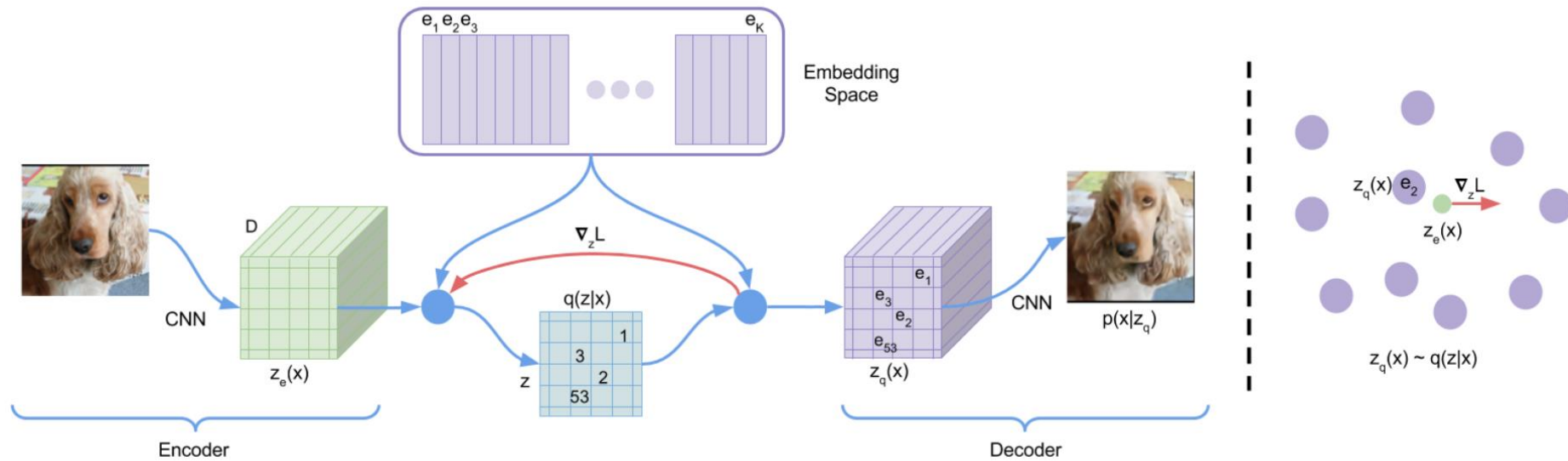
# Nouveau VAE (NVAE)

- Architectural advancement
  - Squeeze and Excitation (SE): channel-wise gating layer



# Vector Quantized VAE (VQ-VAE)

- Powerful decoders (or insufficient data) cause latents being ignored
- Discrete latent embedding space
  - Codebase:  $e \in R^{K \times D}$ , a code:  $e_i \in R^D$
- Posterior categorical distribution  $q(z = k|x) = \begin{cases} 1 & \text{for } k = \operatorname{argmin}_j \|z_e(x) - e_j\|_2, \\ 0 & \text{otherwise} \end{cases}$
- Enforcing categorical distribution for the latent



$$L = \log p(x|z_q(x)) + \|\operatorname{sg}[z_e(x)] - e\|_2^2 + \beta \|z_e(x) - \operatorname{sg}[e]\|_2^2$$

# VQ-VAE

- Code

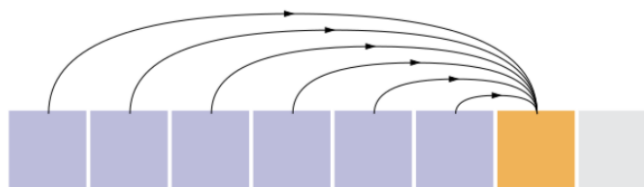
```
z = encoder(img)
# reshape z -> (batch, height, width, channel) and flatten
z = z.permute(0, 2, 3, 1).contiguous()
z_flattened = z.view(-1, e_dim)
d = torch.cdist(z_flattened, embedding.weight)
min_encoding_indices = torch.argmin(d, dim=1)
z_q = embedding(min_encoding_indices)

# compute loss for embedding
loss = torch.mean((z_q.detach() - z) ** 2) + beta * \
| torch.mean((z_q - z.detach()) ** 2)

# preserve gradients (straight-through estimator)
z_q = z + (z_q - z).detach()
reconstructed = decoder(z_q.permute(0, 3, 1, 2).contiguous())
```

# VQ-VAE

- How to sample from the latent?
  - After training the VAE part, we can fit an autoregressive model over the latent
    - Using a raster order
  - Use ancestral sampling to sample  $z$
  - The original VQ-VAE used PixelCNN
- SOTA implementations
  - Transformers
  - Teacher-forcing loss
  - Examples
    - LlamaGen (3.1B)
    - VQ-GAN



$$\mathbf{y} = (y_1, y_2, \dots, y_T)$$

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^T p(y_t \mid y_{<t}, \mathbf{x})$$

$$\mathcal{L}_{\text{TF}} = - \sum_{t=1}^T \log p_{\theta}(y_t \mid y_{1:t-1}, \mathbf{x})$$